Snap Sync 혁신 기법 분석

개념적 배경

Snap Sync는 Ethereum의 가장 혁신적인 동기화 메커니즘으로, 스냅샷 기반의 빠른 상태 동기화를 제공합니다. 기존 Fast Sync가 헤더 체인을 먼저 구축한 후 상태를 동기화하는 반면, Snap Sync는 스냅샷을 활용하여 계정과 스토리지 데이터를 병렬로 다운로드하고 검증합니다.

설계 필요성

1. **동기화 시간 대폭 단축**: Fast Sync 대비 5-10배 빠른 동기화 속도
2. **네트워크 효율성 극대화**: 압축된 스냅샷 데이터로 대역폭 사용량 최소화
3. **병렬 처리 최적화**: 계정과 스토리지를 동시에 다운로드
4. **검증 강화**: 머클 증명을 통한 강력한 데이터 무결성 보장

다른 접근법과의 비교

- **Fast Sync**: 순차적 헤더-상태 동기화, 느린 속도
- **Full Sync**: 모든 블록 실행, 완전한 검증하지만 매우 느림
- **Light Sync**: 헤더만 동기화, 빠르지만 제한적 기능
- **Snap Sync**: 스냅샷 기반 병렬 동기화, 최적의 속도와 보안

구현 분석

핵심 데이터 구조

Syncer 구조체

```go
type Syncer struct {
db ethdb.KeyValueStore // 상태 데이터베이스
scheme  string  // 트라이 스키마 (hash/path)
root common.Hash // 동기화할 상태 루트
// 동기화 작업 관리
tasks  map[string]*accountTask // 계정 동기화 작업
snapped  map[common.Hash][]byte  // 스냅샷된 계정 데이터
// 병렬 처리 제어
accountReqs  map[uint64]*accountRequest // 진행 중인 계정 요청
storageReqs  map[uint64]*storageRequest // 진행 중인 스토리지 요청
bytecodeReqs  map[uint64]*bytecodeRequest // 진행 중인 바이트코드 요청
// 성능 통계
statelessPeers  map[string]struct{} // 상태 없는 피어 목록
rates *msgrate.Tracker // 메시지 전송률 추적
}
```

계정 작업 구조체

```go
type accountTask struct {
// 작업 범위 정의
origin common.Hash // 시작 계정 해시
limit common.Hash // 종료 계정 해시
// 동기화 상태
```

```
req *accountRequest // 현재 요청
res *accountResponse // 응답 데이터
// 하위 작업
SubTasks  map[common.Hash]*storageTask // 스토리지 작업들
// 진행 상황
done  bool  // 작업 완료 여부
}
```

스냅샷 프로토콜 메시지

```
// 계정 범위 요청
type GetAccountRangePacket struct {
ID  uint64  // 요청 ID
Root common.Hash // 상태 루트
Origin common.Hash // 시작 해시
Limit common.Hash // 종료 해시
Bytes  uint64  // 응답 크기 제한
}

// 계정 범위 응답
type AccountRangePacket struct {
ID  uint64  // 요청 ID
Accounts []*AccountData // 계정 데이터 배열
```

```
Proof [][]byte  // 머클 증명
}

// 스토리지 범위 요청
type GetStorageRangesPacket struct {
ID  uint64  // 요청 ID
Root common.Hash // 상태 루트
Accounts []common.Hash // 대상 계정들
Origin []byte  // 시작 슬롯
Limit []byte  // 종료 슬롯
Bytes  uint64  // 응답 크기 제한
}
```

주요 알고리즘

1. 해시 범위 분할 알고리즘

Snap Sync는 전체 계정 공간을 여러 범위로 분할하여 병렬 다운로드를 수행합니다:

```
// hashRange는 해시 공간을 처리하는 유틸리티
type hashRange struct {
current *uint256.Int // 현재 위치
step *uint256.Int // 단계 크기
}
```

```go
// newHashRange는 새로운 해시 범위를 생성
func newHashRange(start common.Hash, num uint64) *hashRange {
// 남은 공간 계산
left := new(big.Int).Sub(hashSpace, start.Big())
// 단계 크기 계산 (균등 분할)
step := new(big.Int).Div(
new(big.Int).Add(left, new(big.Int).SetUint64(num-1)),
new(big.Int).SetUint64(num),
)
return &hashRange{
current: new(uint256.Int).SetBytes32(start[:]),
step: step256,
}
}


// Next는 다음 범위로 이동
func (r *hashRange) Next() bool {
next, overflow := new(uint256.Int).AddOverflow(r.current, r.step)
if overflow {
return false
}
r.current = next
return true
```

```
}
```

2. 병렬 계정 동기화 알고리즘

```
// 계정 동기화 시작
func (s *Syncer) syncAccounts(root common.Hash) error {
// 계정 공간을 동시성 수만큼 분할
ranges := make([]*hashRange, accountConcurrency)
for i := 0; i < accountConcurrency; i++ {
ranges[i] = newHashRange(common.Hash{}, uint64(accountConcurrency))
for j := 0; j < i; j++ {
ranges[i].Next()
}
}
// 각 범위에 대해 병렬 작업 시작
var wg sync.WaitGroup
for i, r := range ranges {
wg.Add(1)
go func(index int, hashRange *hashRange) {
defer wg.Done()
s.syncAccountRange(root, hashRange)
}(i, r)
}
wg.Wait()
```

```
return nil
}
```

3. 스토리지 동기화 알고리즘

각 계정의 스토리지는 별도로 동기화됩니다:

```
// 스토리지 동기화
func (s *Syncer) syncStorage(account common.Hash, root common.Hash) error {
// 대형 컨트랙트의 경우 스토리지도 분할
if s.isLargeContract(account) {
return s.syncStorageRanges(account, root)
}
// 소형 컨트랙트는 한 번에 동기화
return s.syncStorageComplete(account, root)
}

// 대형 컨트랙트 스토리지 범위 동기화
func (s *Syncer) syncStorageRanges(account common.Hash, root common.Hash) error
ranges := make([]*hashRange, storageConcurrency)
for i := 0; i < storageConcurrency; i++ {
ranges[i] = newHashRange(common.Hash{}, uint64(storageConcurrency))
for j := 0; j < i; j++ {
ranges[i].Next()
```

```
}

}

// 병렬 스토리지 동기화

var wg sync.WaitGroup

for _, r := range ranges {

wg.Add(1)

go func(hashRange *hashRange) {

defer wg.Done()

s.syncStorageRange(account, root, hashRange)

}(r)

}

wg.Wait()

return nil

}
```

4. 머클 증명 검증 알고리즘

모든 다운로드된 데이터는 머클 증명으로 검증됩니다:

```
// 계정 범위 검증

func (s *Syncer) verifyAccountRange(req *GetAccountRangePacket,

res *AccountRangePacket) error {

// 응답 데이터 언팩

hashes, accounts, err := res.Unpack()

if err != nil {
```

```
return err
}
// 머클 증명 검증
keys := make([][]byte, len(hashes))
vals := make([][]byte, len(accounts))
for i := 0; i < len(hashes); i++ {
keys[i] = hashes[i][:]
vals[i] = accounts[i]
}
// 트라이 증명 검증
_, err = trie.VerifyRangeProof(req.Root, req.Origin[:], req.Limit[:],
keys, vals, res.Proof)
return err
}
```

코드 구조 및 모듈 관계

```
eth/protocols/snap/
├── sync.go # 메인 동기화 로직
├── protocol.go # 프로토콜 메시지 정의
├── handler.go # 메시지 처리 핸들러
├── range.go # 해시 범위 처리
├── tracker.go # 진행 상황 추적
```

```
└── peer.go # 피어 관리


core/state/snapshot/
├── snapshot.go # 스냅샷 인터페이스
├── generate.go # 스냅샷 생성
├── disklayer.go # 디스크 레이어
├── difflayer.go # 차이 레이어
└── iterator.go # 스냅샷 반복자
```

모듈 간 의존성:

- `sync.go` → 동기화 엔진의 핵심
- `protocol.go` → 네트워크 메시지 정의
- `handler.go` → 피어 통신 처리
- `snapshot/` → 로컬 스냅샷 관리

실행 흐름

단계별 실행 과정

1단계: 동기화 초기화

```go
func (s *Syncer) Sync(root common.Hash, cancel chan struct{}) error {
// 동기화 상태 초기화
s.root = root
s.tasks = make(map[string]*accountTask)
```

```
s.snapped = make(map[common.Hash][]byte)
// 계정 작업 생성
s.createAccountTasks()
// 동기화 시작
return s.run(cancel)
}
```

2단계: 계정 작업 생성 및 분배

```
func (s *Syncer) createAccountTasks() {
// 전체 계정 공간을 16개 범위로 분할
for i := 0; i < accountConcurrency; i++ {
origin := s.calculateRangeStart(i)
limit := s.calculateRangeEnd(i)
task := &accountTask{
origin: origin,
limit: limit,
SubTasks: make(map[common.Hash]*storageTask),
}
s.tasks[fmt.Sprintf("account-%d", i)] = task
}
}
```

3단계: 병렬 계정 다운로드

```
func (s *Syncer) run(cancel chan struct{}) error {
for {
select {
case <-cancel:
return ErrCancelled
default:
// 대기 중인 작업 확인
if task := s.nextAccountTask(); task != nil {
// 계정 범위 요청 전송
s.requestAccountRange(task)
}
// 응답 처리
s.processResponses()
// 완료 확인
if s.allTasksComplete() {
return nil
}
}
}
}
```

4단계: 계정 데이터 처리

```
func (s *Syncer) processAccountResponse(res *accountResponse) error {
// 머클 증명 검증
if err := s.verifyAccountRange(res.req, res); err != nil {
return err
}
// 계정 데이터 저장
batch := s.db.NewBatch()
for i, hash := range res.hashes {
account := res.accounts[i]
// 계정 데이터 저장
rawdb.WriteAccountSnapshot(batch, hash, account)
// 스토리지가 있는 경우 스토리지 작업 생성
if !account.Root.Equal(emptyRoot) {
s.createStorageTask(hash, account.Root)
}
}
return batch.Write()
}
```

5단계: 스토리지 동기화

```
func (s *Syncer) processStorageTask(task *storageTask) error {
// 스토리지 범위 요청
```

```
req := &GetStorageRangesPacket{
ID: s.nextRequestID(),
Root: s.root,
Accounts: []common.Hash{task.account},
Bytes: maxRequestSize,
}
// 피어에게 요청 전송
peer := s.selectBestPeer()
return peer.RequestStorageRanges(req)
}
```

6단계: 힐링 프로세스

```
func (s *Syncer) healState() error {
// 누락된 트라이 노드 식별
missing := s.findMissingTrieNodes()
// 누락된 노드 요청
for _, node := range missing {
req := &GetTrieNodesPacket{
ID: s.nextRequestID(),
Root: s.root,
Paths: [][]byte{node.path},
Bytes: maxRequestSize,
```

```
}

peer := s.selectBestPeer()

peer.RequestTrieNodes(req)

}

return  nil

}
```

상태 변화 추적

Snap Sync 과정에서의 주요 상태 변화:

1. **Idle → Syncing**: 동기화 시작
2. **Account Sync**: 계정 데이터 병렬 다운로드
3. **Storage Sync**: 스토리지 데이터 병렬 다운로드
4. **Bytecode Sync**: 컨트랙트 바이트코드 다운로드
5. **Healing**: 누락된 트라이 노드 복구
6. **Completed**: 동기화 완료

에러 처리 메커니즘

주요 에러 타입

```
var (

ErrCancelled = errors.New("sync cancelled")

ErrInvalidProof = errors.New("invalid merkle proof")

ErrMissingData = errors.New("missing account or storage data")
```

```
ErrBadPeer = errors.New("peer provided invalid data")

)
```

에러 복구 전략

1. **증명 실패**: 해당 피어 드롭 후 다른 피어에서 재요청
2. **데이터 누락**: 힐링 프로세스에서 누락 데이터 복구
3. **네트워크 오류**: 자동 재시도 및 피어 재선택
4. **타임아웃**: 동적 타임아웃 조정 및 재요청

성능 최적화

적용된 최적화 기법

1. 동적 요청 크기 조정

```
const (

minRequestSize = 64 * 1024  // 최소 요청 크기 (64KB)

maxRequestSize = 512 * 1024  // 최대 요청 크기 (512KB)

)


// 피어 성능에 따른 동적 크기 조정

func (s *Syncer) calculateRequestSize(peer *Peer) uint64 {

rate := s.rates.Capacity(peer.id, time.Second)

size := uint64(rate * 0.1) // 10% of capacity

if  size < minRequestSize {
```

```
return minRequestSize
}
if size > maxRequestSize {
return maxRequestSize
}
return size
}
```

2. 병렬 처리 최적화

```
var (
// 계정 동시성: 16개 병렬 작업
accountConcurrency = 16
// 스토리지 동시성: 16개 병렬 작업
storageConcurrency = 16
)

// CPU 코어 수에 따른 동적 조정
func init() {
if runtime.NumCPU() > 8 {
accountConcurrency = 32
storageConcurrency = 32
}
```

```
}
```

3. 메모리 관리 최적화

```
// 배치 크기 임계값
const batchSizeThreshold = 8 * 1024 * 1024 // 8MB

// 메모리 사용량 모니터링
func (s *Syncer) checkMemoryUsage() {
var m runtime.MemStats
runtime.ReadMemStats(&m)
// 메모리 사용량이 높으면 배치 크기 감소
if m.Alloc > 1024*1024*1024 { // 1GB
s.reduceBatchSize()
}
}
```

4. 네트워크 최적화

```
// 피어 성능 기반 작업 할당
func (s *Syncer) selectBestPeer() *Peer {
var best *Peer
var bestRate float64
for _, peer := range s.peers {
```

```
rate := s.rates.Capacity(peer.id, time.Second)

if  rate > bestRate {

best = peer

bestRate = rate

}

}

return  best

}
```

병목 지점 분석

1. 네트워크 병목

- **문제**: 느린 피어로 인한 전체 동기화 지연
- **해결**: 피어 성능 모니터링 및 동적 재할당

2. 검증 병목

- **문제**: 머클 증명 검증으로 인한 CPU 부하
- **해결**: 병렬 검증 및 검증 결과 캐싱

3. 디스크 I/O 병목

- **문제**: 대량의 스냅샷 데이터 쓰기
- **해결**: 배치 쓰기 및 압축 사용

튜닝 가능한 매개변수

동시성 관련 매개변수

```
// 조정 가능한 동시성 설정
var (
accountConcurrency = 16  // 계정 병렬 작업 수
storageConcurrency = 16  // 스토리지 병렬 작업 수
maxCodeRequestCount = 85  // 바이트코드 요청 수
maxTrieRequestCount = 1024  // 트라이 노드 요청 수
)
```

성능 튜닝 가이드

1. **고성능 환경**: 동시성 수치 증가 (32-64)
2. **저사양 환경**: 동시성 수치 감소 (8-16)
3. **네트워크 최적화**: 요청 크기 조정
4. **메모리 최적화**: 배치 크기 제한

실제 사용 예시

코드 예제

Snap Sync 시작

```
// Snap Sync 시작
func startSnapSync(syncer *snap.Syncer, root common.Hash) error {
// 취소 채널 생성
cancel := make(chan struct{})
```

```
// 동기화 시작
go func() {
err := syncer.Sync(root, cancel)
if err != nil {
log.Error("Snap sync failed", "err", err)
} else {
log.Info("Snap sync completed successfully")
}
}()
return nil
}
```

진행 상황 모니터링

```
// Snap Sync 진행 상황 모니터링
func monitorSnapSync(syncer *snap.Syncer) {
ticker := time.NewTicker(5 * time.Second)
defer ticker.Stop()
for {
select {
case <-ticker.C:
progress, pending := syncer.Progress()
fmt.Printf("계정 동기화: %d/%d (%.2f%%)\n",
progress.AccountSynced,
```

```
progress.AccountSynced + pending.Accounts,
float64(progress.AccountSynced)/float64(progress.AccountSynced + pending.Account
fmt.Printf("스토리지 동기화: %d 슬롯\n", progress.StorageSynced)
fmt.Printf("바이트코드 동기화: %d 컨트랙트\n", progress.BytecodeSynced)
if  pending.Accounts == 0 && pending.Storage == 0 {
fmt.Println("Snap Sync 완료!")
return
}
}
}
}
```

설정 방법

geth 명령행 옵션

```
# Snap Sync 모드로 시작 (기본값)
geth  --syncmode  snap

# 동시성 조정 (내부적으로 자동 조정됨)
geth  --syncmode  snap  --cache  2048

# 스냅샷 비활성화 (Fast Sync로 전환)
```

```
geth  --syncmode  snap  --snapshot=false
```

프로그래밍 방식 설정

```
// Snap Sync 설정
config := &ethconfig.Config{
SyncMode: ethconfig.SnapSync,
SnapshotCache: 256,
DatabaseCache: 1024,
}

// 스냅샷 활성화
config.NoPruning = false
config.TrieCleanCache = 256
config.TrieDirtyCache = 512
```

문제 해결 가이드

일반적인 문제와 해결 방법

1. **동기화 중단**

- 원인: 피어 연결 문제 또는 잘못된 데이터
- 해결: 피어 재연결 및 힐링 프로세스 실행

2. **검증 실패**

- 원인: 손상된 머클 증명 또는 악의적 피어

- 해결: 해당 피어 차단 및 다른 피어에서 재다운로드

3. **메모리 부족**

- 원인: 과도한 병렬 작업
- 해결: 동시성 수치 감소 또는 메모리 증설

4. **디스크 공간 부족**

- 원인: 스냅샷 데이터 크기
- 해결: 충분한 디스크 공간 확보

로그 분석

```
// Snap Sync 관련 로그 모니터링
log.Info("Snap sync started", "root", root, "accounts", accountTasks)
log.Info("Account sync progress", "synced", synced, "pending", pending)
log.Info("Storage sync progress", "slots", slots, "contracts", contracts)
log.Info("Healing progress", "nodes", healedNodes, "missing", missingNodes)
```

관련 시스템

의존성 관계

상위 의존성

- **eth/downloader**: 전체 동기화 조정
- **eth/handler**: 피어 메시지 처리
- **core/blockchain**: 블록체인 상태 관리

하위 의존성

- **core/state/snapshot**: 로컬 스냅샷 관리
- **trie**: 머클 트리 검증
- **ethdb**: 데이터베이스 접근

상호작용 패턴

스냅샷 시스템과의 상호작용

```
// 로컬 스냅샷 생성
func (s *Syncer) createSnapshot(root common.Hash) *snapshot.Tree {
return  snapshot.New(s.db, s.triedb, s.cache, root, true)
}

// 스냅샷 데이터 조회
func (s *Syncer) getAccountFromSnapshot(hash common.Hash) (*types.StateAccount,
return  s.snapshot.Account(hash)
}
```

P2P 네트워킹과의 상호작용

```
// 피어에게 계정 범위 요청
func (p *Peer) RequestAccountRange(req *GetAccountRangePacket) error {
return  p2p.Send(p.rw, GetAccountRangeMsg, req)
}
```

```
// 스토리지 범위 요청
func (p *Peer) RequestStorageRanges(req *GetStorageRangesPacket) error {
return  p2p.Send(p.rw, GetStorageRangesMsg, req)
}
```

트라이 시스템과의 상호작용

```
// 머클 증명 검증
func (s *Syncer) verifyProof(root common.Hash, keys [][]byte,
vals [][]byte, proof [][]byte) error {
_, err := trie.VerifyRangeProof(root, keys[0], keys[len(keys)-1],
keys, vals, proof)
return  err
}
```

영향 범위

직접적 영향

- **동기화 성능**: 전체 노드 시작 시간을 5-10배 단축
- **네트워크 효율성**: 대역폭 사용량 최적화
- **사용자 경험**: 빠른 노드 준비 시간

간접적 영향

- **네트워크 참여도**: 낮은 진입 장벽으로 더 많은 노드 참여

- **개발자 생산성**: 빠른 테스트 환경 구축
- **생태계 건강성**: 분산화 수준 향상

혁신적 특징

1. 스냅샷 기반 동기화

- **기존 방식**: 순차적 블록 다운로드 및 실행
- **Snap Sync**: 압축된 상태 스냅샷 직접 다운로드

2. 병렬 처리 아키텍처

- **계정 병렬화**: 16개 범위로 분할하여 동시 다운로드
- **스토리지 병렬화**: 대형 컨트랙트 스토리지도 병렬 처리
- **검증 병렬화**: 머클 증명 검증을 별도 스레드에서 수행

3. 적응형 힐링 메커니즘

- **누락 감지**: 자동으로 누락된 트라이 노드 식별
- **선택적 복구**: 필요한 노드만 선별적으로 다운로드
- **점진적 완성**: 동기화 중에도 지속적으로 상태 완성

4. 강화된 보안 모델

- **범위 증명**: 각 데이터 범위에 대한 머클 증명 제공
- **무결성 검증**: 모든 데이터의 암호학적 검증
- **악의적 피어 방어**: 잘못된 데이터 제공 시 자동 피어 차단

Snap Sync는 Ethereum 동기화 기술의 혁신적 발전으로, 스냅샷 기반 병렬 처리를 통해 동기화 시간을 대폭 단축하면서도 강력한 보안을 제공합니다. 이는 Ethereum 네트워크의 접근성을 크게 향상시

키고, 더 많은 사용자가 풀 노드를 운영할 수 있게 하는 핵심 기술입니다.